



Electrical Engineering Association

Deep Neural Network Compression & Computer Vision

Indian Institute of Technology Kanpur

December 2025 - January 2026

Mentors

Krish Jain | Ayush Jha | Sahil Pandey

Mentees

Reebal Faakhir
Nandini Yadav
Rushikesh Musale

Aditya Singh
Naivedya Jain
Priyanshu Joshi

Kartik Basant
Krish Mesariya
Darshan

June 2025

Abstract

This report presents a structured study of digital image processing, computer vision, and introductory machine learning concepts developed through a progressive learning pipeline. It covers image representation, spatial and frequency-domain analysis, convolution, Fourier transforms, and frequency-based filtering, followed by histogram analysis, color spaces, and intensity transformations. Classical vision techniques such as blurring, edge detection, sharpening, gradient and corner analysis, and Hough-based shape detection are examined from both mathematical and practical perspectives.

The report further explores feature engineering using color, shape, and texture descriptors, demonstrating how handcrafted features can effectively encode visual information for learning tasks. Machine learning foundations are introduced through K-Means clustering, linear and logistic regression, with emphasis on loss functions, optimization, overfitting, and regularization. Finally, neural network concepts including Multilayer Perceptrons, automatic differentiation, and Convolutional Neural Networks are discussed to illustrate the transition from traditional computer vision to learning-based models. Overall, the work highlights how classical vision techniques and modern machine learning methods complement each other in building effective visual systems.

Contents

1	Digital Image Representation	1
1.1	RGB Images and Image as Matrix Representation	1
1.2	Grayscale Images	1
2	The Fourier Domain	2
2.1	The 2D DFT Formula	2
2.2	Fast Fourier Transform (FFT)	2
3	Magnitude and Phase Analysis	2
3.1	Magnitude Spectrum	2
3.2	Phase Spectrum	2
4	Frequency-Domain Filtering	3
5	Histograms	3
5.1	Grayscale Histograms	3
5.2	RGB Histograms	4
6	Color Spaces	4
6.1	HSV (Hue, Saturation, Value)	4
6.2	HSL (Hue, Saturation, Lightness)	5
7	Basic Image Transformations	5
7.1	Brightness	5
7.2	Contrast	5
7.3	Saturation	6
7.4	Hue	6
8	Advanced Color Manipulation	6
8.1	Gamma Correction (Non-linear Transformation)	6
8.2	White Balance (Gray World Algorithm)	7
9	Vibrance vs. Saturation	7
9.1	Saturation	7
9.2	Vibrance (Selective Saturation)	7
10	Image Convolution	8
10.0.1	Properties of Kernels and their Functionalities	8
11	Applications of Image Convolution	8
11.1	Image Blurring	8
11.1.1	Average Blur	8
11.1.2	Gaussian Blur	9

11.2	Edge Detection	9
11.2.1	Image Gradients	9
11.2.2	Sobel Edge Detection	9
11.2.3	Canny Edge Detection	10
12	Image Sharpening	10
12.1	Laplacian Sharpening	11
12.2	Unsharp Masking	11
13	Key observations	11
14	Image Gradients and Corners	11
14.1	Theory	11
14.2	Mathematical Formulation	12
14.3	Visual Results	12
15	Color Spaces	12
15.1	RGB Histogram	12
15.2	HSV Color Space	12
15.3	HSL Color Space	12
16	Hough Transform	12
16.1	Line Detection	12
16.2	Alternative Parameterization	13
16.3	Algorithm (Line Hough Transform)	13
17	Circle and Ellipse Detection	13
17.1	Circle Detection	13
17.2	Ellipse Detection	13
18	Brightness, Contrast, and Saturation	13
18.1	Brightness	13
18.2	Contrast	14
18.3	Saturation	14
19	Key Observations and Takeaways	14
20	Feature Engineering and first group assignment	14
20.1	Color Features	14
20.1.1	Mean Hue	14
20.1.2	Standard Deviation of Hue	15
20.1.3	Mean Saturation	15
20.1.4	Standard Deviation of Saturation	15
20.1.5	Mean Value	15
20.1.6	Standard Deviation of Value	15

20.2	Shape Features	15
20.2.1	Area Ratio	15
20.2.2	Aspect Ratio	15
20.2.3	Solidity	16
20.2.4	Circularity	16
20.3	Hu Moments	16
20.3.1	Why Only the First Two Moments Were Used	16
20.3.2	Interpretation of Hu Moments	16
20.4	Texture Features: Local Binary Patterns (LBP)	16
20.4.1	Intuition Behind LBP	17
20.4.2	Why LBP Is Useful	17
20.4.3	LBP Histogram	17
20.5	Final Feature Pipeline Summary	17
20.6	Key Takeaway from Session 5	17
20.7	Object-Oriented Programming (OOPS), Data Loader and first group as- signment	17
20.7.1	Dataset Class as an OOPS Abstraction	17
20.7.2	DataLoader Class and Controlled Data Flow	18
20.7.3	Integration Through OOPS and Practical Application	18
21	Unsupervised Learning: K-Means Clustering	19
21.1	Problem Formulation	19
21.2	Objective Function	19
21.3	Algorithmic Procedure	19
21.4	Initialization Sensitivity and K-Means++	19
22	Supervised Learning: Linear Regression	19
22.1	Model Assumptions	19
22.2	Loss Function and Maximum Likelihood Estimation	20
22.3	Gradient-Based Optimization	20
22.4	Overfitting and Polynomial Feature Expansion	20
22.5	Regularization	20
23	Supervised Learning: Logistic Regression	20
23.1	Classification Setting	20
23.2	Sigmoid Activation	21
23.3	Likelihood and Loss Function	21
24	MLP and PyTorch Library	21
24.1	Definition and Dimensionality	21
24.2	The Hardware Advantage: GPU Acceleration	23
24.2.1	The “Impossible Calculus” Problem	24
24.2.2	PyTorch vs TensorFlow: Computational Graph Design	24
24.2.3	Dynamic Computational Graphs in PyTorch	25

24.2.4	Backward Pass and the Chain Rule	25
24.2.5	Controlling Gradient Computation in PyTorch	25
24.3	Multi-Layer Perceptrons (MLPs)	25
24.3.1	Limitations of Linear Models	25
24.3.2	Hidden Layers and Representation Learning	25
24.3.3	Activation Functions	26
24.3.4	The Learning Loop in MLPs	26
24.4	Overfitting and Regularization in Neural Networks	26
24.4.1	The Overfitting Problem	26
24.4.2	Bias–Variance Tradeoff	26
24.4.3	Regularisation	26
24.5	Threat Zone Finding Assignment	27
25	Convolutional Neural Networks	27
25.1	Convolutional Layers	27
25.1.1	Concept and Motivation	27
25.1.2	Mathematical Formulation	27
25.1.3	Geometric and Computational Intuition	28
25.2	Pooling Operations	28
25.2.1	Purpose of Pooling	28
25.2.2	Mathematical Definition	28
25.2.3	Intuition and Effect	29
25.3	Fully Connected Layers	29
25.3.1	Role in Neural Networks	29
25.3.2	Mathematical Formulation	29
25.3.3	Comparison with Convolutional Layers	29
25.3.4	End-to-End Perspective	29

1 Digital Image Representation

An **image** is a visual representation or likeness of an object, formed by light interacting with an optical system or stored electronically as pixels. In digital processing, we primarily focus on:

1.1 RGB Images and Image as Matrix Representation

Digital images are commonly represented in the RGB color model, where each image is composed of three separate channels corresponding to Red (R), Green (G), and Blue (B) intensity components. Each channel captures the contribution of its respective color at every pixel location.

Mathematically, an RGB image can be viewed as a three-dimensional array:

$$\mathbf{I} \in \mathbb{R}^{M \times N \times 3}$$

where M and N denote the image height and width, and the third dimension represents the three color channels. Each pixel is thus represented as a vector:

$$\mathbf{I}(x, y) = [R(x, y), G(x, y), B(x, y)]$$

Each color channel itself can be interpreted as a two-dimensional matrix:

$$R, G, B \in \mathbb{R}^{M \times N}$$

where the matrix entries correspond to intensity values, typically in the range $[0, 255]$ for 8-bit images or normalized to $[0, 1]$.



This matrix-based representation enables the application of linear algebra and signal processing techniques to images. Operations such as filtering, transformation, and compression can be performed either independently on each channel or jointly across channels. Grayscale images can be derived from RGB images by combining the three channels into a single intensity matrix, which is discussed in the subsequent section.

1.2 Grayscale Images

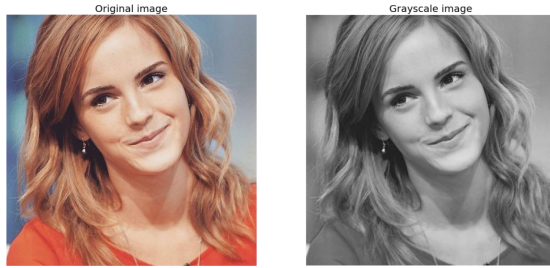
A grayscale image consists of shades ranging from pure black to pure white.

- **Matrix Structure:** It is a 2D matrix (or array) where each element (pixel) holds a single intensity value.
- **Intensity Range:** Typically, values range from 0 (black) to 255 (white).
- **Average Method:** This treats all color channels as having equal importance:

$$I_{GRAY} = \frac{R + G + B}{3}$$

- **Luminosity (Weighted) Method:** Accounts for human vision sensitivity to green light:

$$I_{GRAY} = 0.299(R) + 0.587(G) + 0.114(B)$$



2 The Fourier Domain

The **Fourier Series** breaks down periodic patterns into a combination of simple oscillations, such as sine waves. In digital data, we use the **Discrete Fourier Transform (DFT)**.

2.1 The 2D DFT Formula

For an image $f(x, y)$ of size $M \times N$, the transform $F(u, v)$ is defined as:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

The result $F(u, v)$ is a grid of complex numbers: $F(u, v) = \text{Re}(u, v) + j \cdot \text{Im}(u, v)$.

2.2 Fast Fourier Transform (FFT)

Because the standard DFT requires $O(n^2)$ operations, the **FFT** is used as an efficient algorithm to achieve the same result in $O(n \log n)$ time.

3 Magnitude and Phase Analysis

The frequency domain is analyzed through two distinct spectra:

3.1 Magnitude Spectrum

Describes the strength or "energy" of each frequency component.

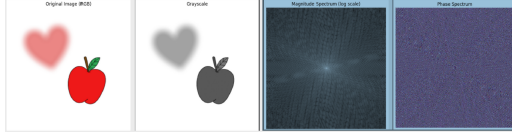
- **Formula:** $|F(u, v)| = \sqrt{\text{Re}(u, v)^2 + \text{Im}(u, v)^2}$
- **Visualization:** Because low frequencies are orders of magnitude stronger, logarithmic scaling is used:

$$\text{Mag} = c \cdot \log(1 + |F(u, v)|)$$

3.2 Phase Spectrum

Contains the spatial structure, including edges, outlines, and shapes.

- **General Formula:** $\theta = \arctan\left(\frac{\Im(F(u, v))}{\Re(F(u, v))}\right)$
- **Limitation of arctan:** The basic arctangent computes the angle using only the ratio $\Im(F)/\Re(F)$, which causes quadrant ambiguity and becomes undefined when $\Re(F) = 0$, making it unsuitable for reliable phase computation.
- **Phase Formula Using atan2:** $\phi(u, v) = \text{atan2}(\Im(F(u, v)), \Re(F(u, v)))$
- **Why atan2 is used:** The `atan2` function explicitly accounts for the signs of both the imaginary and real components, ensuring correct phase estimation across all four quadrants and providing numerical stability when the real part approaches zero.
- **Significance:** Phase encodes the spatial structure and positional information of image features. Even with magnitude removed, the phase component alone can reconstruct perceptually meaningful image structures, commonly summarized as **Phase** $\gg$ **Magnitude**.



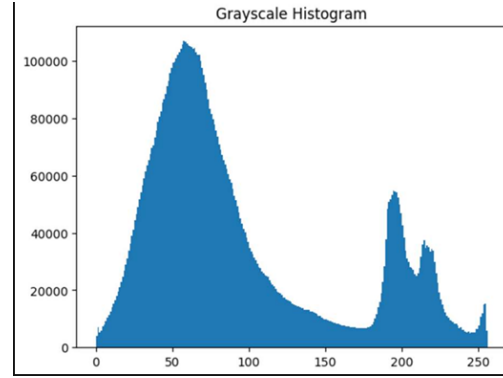
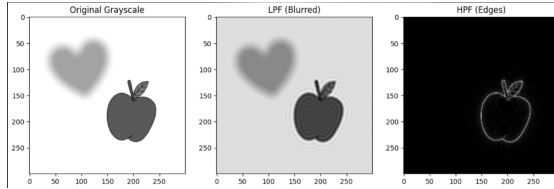
On the Y-axis, we plot the pixel count, and on the X-axis, we plot the intensity ranging from 0 (meaning completely black pixels) to 255 (meaning completely white pixels).

4 Frequency-Domain Filtering

Filtering involves modifying the Fourier Transform by a Filter Mask $H(u, v)$:

$$G(u, v) = H(u, v) \cdot F(u, v)$$

- **Low Pass Mask:** $H(u, v) = 1$ if $D(u, v) \leq D_0$, else 0.
- **High Pass Mask:** $H(u, v) = 1$ if $D(u, v) > D_0$, else 0.



Interpretation:

- **Left-skewed/leaning histogram:** Signifies that the majority of the pixels are in the left half of the scale; that is, most pixel intensity values are closer to 0 than 255. It indicates a **dark image**.
- **Centered histogram:** Signifies that the majority of the pixels are in the middle of the scale. It indicates a **balanced exposure**.
- **Right-skewed/leaning histogram:** Signifies that the majority of the pixels are in the right half of the scale; that is, most pixel intensity values are closer to 255 than 0. It indicates a **bright image**.
- **Narrow spread:** Signifies the range of the intensity values for pixels all over the image is low. Indicates **low contrast** (pixel distinctness is low).
- **Wide spread:** Signifies the range of the intensity values for pixels all over the image is high. Indicates **high contrast** (full dynamic range is utilized).

5 Histograms

An image histogram is a graphical representation of the tonal distribution in a digital image. It is a plot showing the number of pixels at each intensity value, where the intensity values range from 0 to 255.

5.1 Grayscale Histograms

For a single-channel grayscale image, the possible intensity values range from 0 to 255. Mathematically, let X be an image with intensity levels I_k where $k = 0, 1, \dots, 255$. The histogram $h(I_k)$ is defined as the discrete function:

$$h(I_k) = n_k$$

Where n_k is the count of pixels in the image with intensity I_k .

5.2 RGB Histograms

As we know, an RGB image consists of three separate channels, each for R (Red), G (Green), and B (Blue). Thus, an RGB histogram consists of three separate histograms, one for each color channel: $h_R(I_{rk})$, $h_G(I_{gk})$, and $h_B(I_{bk})$.

Interpretation: These histograms reveal the global color balance of the image.

- **High B-Channel values:** If the blue histogram is leaning significantly to the right compared to red and green (i.e., has high B-channel values), the image likely has a **cool color temperature**.
- **High R-Channel values:** If the red histogram is leaning significantly to the right compared to blue and green (i.e., has high R-channel values), the image likely has a **warm color temperature**.
- **Unequal Peaks:** Signifies high **color imbalance** in the image.

6 Color Spaces

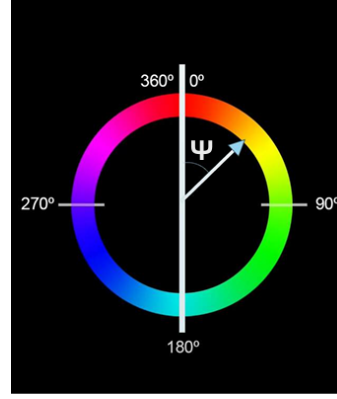
Color spaces are mathematical models used to represent colors as tuples of numbers. While RGB is the standard for hardware (displays/sensors), it may not always be intuitive for human perception or some image processing tasks.

6.1 HSV (Hue, Saturation, Value)

HSV is a cylindrical coordinate representation of points in an RGB color model. It rearranges the geometry of RGB to be more like a human perception-based view.

- **Hue (H) or Chroma:** It is the purest form of a color when it has 100% saturation and 100% brightness. Represents the dominant wave-

length (color type) measured as an angle $\in [0^\circ, 360^\circ]$. Red starts at 0° . In OpenCV, it is scaled down so that the values range from 0 to 180 degrees, due to the 8-bit number representation constraints.



- **Saturation (S):** Represents how vivid a color is; that is, the purity of the color (distance from the center axis as a measure of how much gray is mixed in). Ranges from 0 (gray) to 100% (pure color). In OpenCV, it is represented as a number ranging from 0 to 255.
- **Value (V):** Represents the brightness or intensity of the color. Ranges from 0 (black) to 100% (white). In OpenCV, it is represented as a number ranging from 0 to 255.

Mathematical Conversion (RGB $\rightarrow$ HSV): Given $R, G, B \in [0, 1]$: Let $C_{max} = \max(R, G, B)$, $C_{min} = \min(R, G, B)$, and $\Delta = C_{max} - C_{min}$.

Value (V):

$$V = C_{max}$$

Saturation (S):

$$S = \begin{cases} 0 & \text{if } C_{max} = 0 \\ \frac{\Delta}{C_{max}} & \text{if } C_{max} \neq 0 \end{cases}$$

Hue (H):

$$H = \begin{cases} 0^\circ & \text{if } \Delta = 0 \\ 60^\circ \cdot \left(\frac{G-B}{\Delta} \bmod 6 \right) & \text{if } C_{max} = R \\ 60^\circ \cdot \left(\frac{B-R}{\Delta} + 2 \right) & \text{if } C_{max} = G \\ 60^\circ \cdot \left(\frac{R-G}{\Delta} + 4 \right) & \text{if } C_{max} = B \end{cases}$$

6.2 HSL (Hue, Saturation, Lightness)

HSL is similar to HSV but conceptually distinct in its definition of brightness (Lightness vs. Value).

Lightness (L): Defined as the average of the strongest and weakest colors.

$$L = \frac{C_{max} + C_{min}}{2}$$

Lightness chooses the midpoint between black and white; that is, $L = 0$ indicates black, $L = 1$ indicates white, and $L = 0.5$ indicates true color. So, in HSL, saturation is strongest at $L = 0.5$ and weak at extremes.

Implication: In HSV, pure colors (like bright red) have $V = 1$. In HSL, pure colors have $L = 0.5$; only pure white has $L = 1$. This makes HSL symmetrical regarding lightness and darkness compared to HSV.

7 Basic Image Transformations

7.1 Brightness

The brightness of an image refers to the overall intensity or luminance of the pixels in the image. It is a measure of how light or dark the image appears. It corresponds to the **Value** in HSV.

Given an input pixel I_{in} , the output I_{out}

is:

$$I_{out} = I_{in} + \beta$$

- $\beta > 0$: Image becomes lighter. That is, the brightness increases. It shifts the histogram to the right.
- $\beta < 0$: Image becomes darker. That is, the brightness decreases. It shifts the histogram to the left.

We call β the **Brightness Factor**.

Clipping: Since pixel values are bounded (e.g., $[0, 255]$), values exceeding the max are clipped to 255 and values below 0 are clipped to 0.

Brightened Image $_{ij} =$

$$\text{clip}(I_{ij} + \text{Brightness Factor}, 0, 255)$$

7.2 Contrast

Contrast in an image refers to the difference in brightness between different parts of the image. Increasing contrast enhances the difference between the light and dark areas, making the image appear more vivid and distinct. Conversely, decreasing contrast reduces the difference, resulting in a more subdued or flat appearance.

Mathematically, $\text{new} = \alpha \times \text{old}$.

- $\alpha > 1$: Increases contrast (histogram stretches). Dark becomes darker, bright becomes brighter.
- $0 < \alpha < 1$: Decreases contrast (histogram shrinks towards the mean).

$$\text{Contrasted Image} = \text{clip}(\text{Contrast Factor} \times (I_{ij} - \text{midpoint}) + \text{midpoint}, 0, 255)$$

Effects:

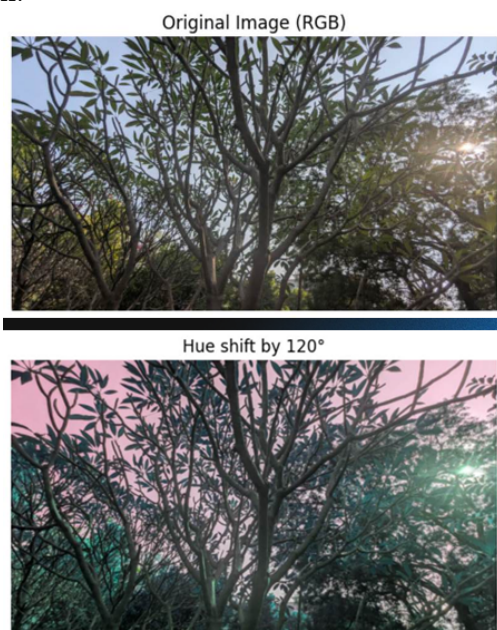
- Increases depth.
- Makes edges sharper.
- Can make shadows lose detail.

7.3 Saturation

Saturation describes the intensity or vividness of the color.

- High saturation $\rightarrow$ vibrant colors.
- Low saturation $\rightarrow$ grayscale/washed-off.
- In HSV: $S = 0 \rightarrow$ grayscale; $S > 0 \rightarrow$ color becomes more pronounced.

When saturation is increased aggressively, the intensity values of specific color channels are multiplied to the point where they exceed the maximum displayable limit (e.g., 255); consequently, these values are abruptly cut off or 'clipped' at the maximum, causing subtle variations in texture to merge into flat, uniform patches of solid color, which destroys the image's fine detail.



7.4 Hue

Hue is the angle on the color wheel. Hue manipulation can be understood as rotating the color wheel.

Examples:

- $+20^\circ$ hue: reds become slightly orange.
- $+90^\circ$ hue: colors shift dramatically (green $\rightarrow$ blue, red $\rightarrow$ cyan, etc.).
- $+120^\circ$ hue: RGB changes to BRG

Hue rotation preserves:

- Brightness
- Saturation
- Texture

Only color identity changes.

8 Advanced Color Manipulation

8.1 Gamma Correction (Non-linear Transformation)

Human vision perceives brightness logarithmically, not linearly (we are more sensitive to changes in dark tones than bright tones). Sensors capture light linearly. Gamma correction maps linear data to non-linear perceptual data.

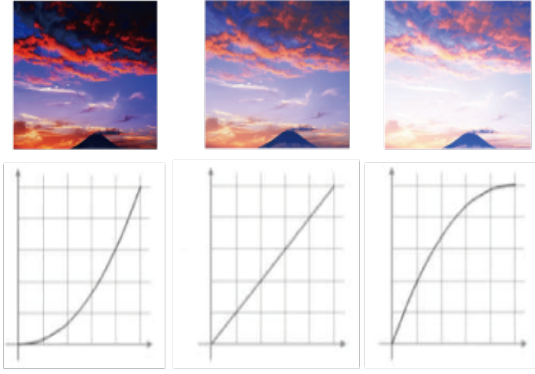
Gamma (γ) is a non-linear exponent used to describe how brightness behaves in images and displays. The transformation follows a power-law relationship:

$$V_{out} = A \cdot V_{in}^\gamma$$

- **Encoding Gamma ($\gamma < 1$):** Compresses bright values to save bandwidth/storage for dark values.
- **Decoding/Display Gamma ($\gamma > 1$):** Standard displays (CRT/LCD) usually follow a gamma of ≈ 2.2 .

Correction Logic: To ensure the display output matches the original scene, we apply the inverse gamma:

$$I_{corrected} = 255 \cdot \left(\frac{I_{in}}{255} \right)^{1/\gamma}$$



8.2 White Balance (Gray World Algorithm)

White balance removes unrealistic color casts so that objects which appear white in person are rendered white in the image.

Goal:

- Neutralize the dominant color in the image (yellow in indoor lighting, towards red when warm).
- Adjust the image to appear as if shot under daylight.
- We can get more neutral skin tones, correct product photos, and more natural colors.

Mathematical Approaches:

- **Gray World Assumption:** Assumes average image color should be gray (equal R, G, B values).
- **White Patch Method:** Assumes brightest pixel should be white and scales accordingly.
- **Learning-Based White Balance:** Adjusts the values with the help of machine learning models.

The Gray World Assumption: Given an image with sufficient color variety, the average color of the scene should be neutral gray.

1. Calculate average intensities for each channel: μ_R, μ_G, μ_B .

2. Calculate the global gray average:

$$\mu_{gray} = \frac{\mu_R + \mu_G + \mu_B}{3}$$

3. Calculate scaling factors for each channel to force their average to μ_{gray} :

$$k_R = \frac{\mu_{gray}}{\mu_R}, \quad k_G = \frac{\mu_{gray}}{\mu_G}, \quad k_B = \frac{\mu_{gray}}{\mu_B}$$

4. Apply transformation:

$$R' = k_R \cdot R, \quad G' = k_G \cdot G, \quad B' = k_B \cdot B$$

9 Vibrance vs. Saturation

While both operations increase color intensity, they behave differently regarding skin tones and already-saturated pixels.

9.1 Saturation

Saturation applies a linear multiplier to the saturation channel (S) in HSV/HSL space.

$$S_{new} = S_{old} \times \text{scale}$$

Drawback: It boosts all pixels equally. Pixels that are already vivid will clip (lose detail), and skin tones often become unnaturally orange/red.

9.2 Vibrance (Selective Saturation)

Vibrance increases the saturation of less-saturated colors while preventing the clipping of already-saturated colors.

Protects:

- Skin tones
- Already-saturated pixels

It is more photographic and subtle. It acts as a non-linear masking function.

Vibrance is NOT standardized, but here's a common implementation:

1. Convert into HSV or HSL.
2. For each pixel:
 - a) Measure current saturation S .
 - b) Apply boosting mainly when S is low:

$$S' = S + \alpha \cdot (1 - S) \cdot S$$

3. Keep hues near skin tone ($\approx 25^\circ - 50^\circ$) protected.

10 Image Convolution

Image convolution is a fundamental operation in computer vision and forms the basis of many image processing techniques, ranging from basic filtering to advanced feature extraction. Convolution applies a small matrix, called a kernel, across an image, where kernel and pixel values are multiplied and summed at each location to produce the output. Repeating this operation across the image generates a transformed output that reflects the properties encoded in the kernel. Formally, the two-dimensional convolution is defined as

$$(I * K)(x, y) = \sum_i \sum_j I(x+i, y+j) K(i, j)$$

where I denotes the input image, K denotes the kernel, and (x, y) denotes the output pixel location. The indices i and j iterate over the dimensions of the kernel. For a common 3×3 kernel, these indices would take the value $-1, 0$ or 1 .

10.0.1 Properties of Kernels and their Functionalities

Convolution kernels are characterized by two key properties.

1. Sum of kernel weights:
 - It determines the effect on image intensity
 - Blur kernels usually have a unit sum, sharpening kernels have sums greater than one or negative, and edge detection kernels are zero-sum.
2. Symmetry of the Kernel
 - It influences the directional sensitivity
 - Gaussian blur kernels are symmetric, whereas Sobel kernels are directional.

:

11 Applications of Image Convolution

11.1 Image Blurring

Image blurring is a preprocessing operation that reduces noise and fine details by averaging pixel values over a local neighborhood. This is majorly done in two ways.

1. Average Blur
2. Gaussian Blur

11.1.1 Average Blur

Average blurring smooths an image by replacing each pixel with the unweighted mean of its neighbors using a uniform kernel. It is effective in reducing noise and softening the image. However, since all pixels are treated equally, it also blurs important edges and fine details. The following kernel is used for this.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

11.1.2 Gaussian Blur

Gaussian blurring performs image smoothing using a weighted average, where weights follow a Gaussian distribution that emphasizes the central pixel and decreases smoothly with distance. This results in more natural smoothing and better edge preservation than average blurring, making it well-suited for image pre-processing before feature extraction. The 2D Gaussian function used to generate these kernels is given by:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

An example of a 3x3 Gaussian kernel derived from this function is:

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

11.2 Edge Detection

11.2.1 Image Gradients

An image gradient quantifies the rate and direction of change in pixel intensity and forms the mathematical basis of edge detection. Since edges correspond to rapid intensity variations, gradient analysis enables effective feature extraction by capturing both edge strength and orientation. Mathematically, the two-dimensional image gradient is defined as:

$$\nabla I = \begin{bmatrix} \frac{\partial I}{\partial x} \\ \frac{\partial I}{\partial y} \end{bmatrix}$$

- **Gradient Magnitude (Edge Strength):** This scalar value indicates how strong the edge is. It is calculated as the Euclidean norm of

the gradient vector:

$$S = \|\nabla I\| = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2}$$

- **Gradient Orientation (Edge Direction):** This value indicates the orientation of the edge, typically measured as an angle relative to the horizontal axis:

$$\theta = \tan^{-1}\left(\frac{\partial I/\partial y}{\partial I/\partial x}\right)$$

11.2.2 Sobel Edge Detection

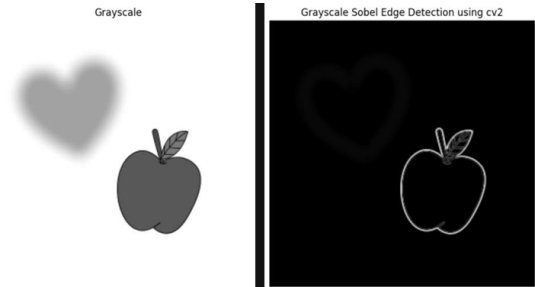
The Sobel operator approximates image gradients using two 3x3 convolution kernels for horizontal (G_x) and vertical (G_y) intensity changes. The image is convolved with each kernel to obtain gradient maps, which are then combined to compute the edge magnitude.

$$G = \sqrt{G_x^2 + G_y^2}$$

where

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$



11.2.3 Canny Edge Detection

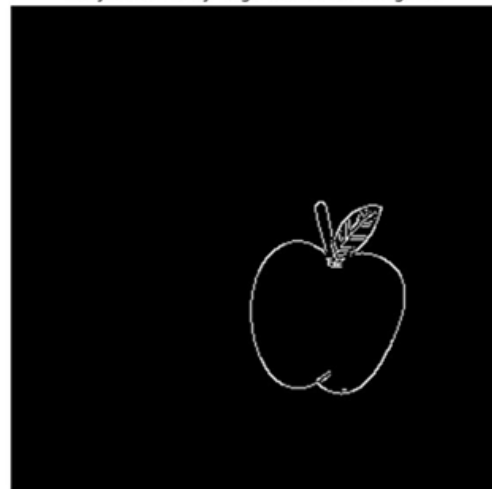
The Canny edge detector is a widely used multi-stage algorithm designed to produce clean, thin, and accurate edges. It consists of the following five key steps.

1. Gaussian smoothing: Applies initial blurring to suppress noise.
2. Gradient computation: Uses Sobel operators to estimate gradient magnitude and direction.
3. Non-maximum suppression: Thins edges by retaining only local maxima along the gradient direction.
4. Double thresholding: Classifies pixels as strong, weak, or non-edges using low and high thresholds.
5. Edge tracking by hysteresis: Preserves weak edges connected to strong edges while discarding isolated responses.

Grayscale



Grayscale Canny Edge Detection using cv2



The edges produced here are significantly cleaner, thinner, and more accurate than those produced by the Sobel operator alone.

12 Image Sharpening

Image sharpening enhances image clarity by emphasizing high-frequency components such as edges and fine details. By amplifying features that are typically reduced by blurring, sharpening makes images appear crisper and more visually distinct.

12.1 Laplacian Sharpening

The Laplacian is a second-order derivative operator that is more sensitive to rapid intensity changes than first-order operators such as Sobel. This property makes it effective for enhancing fine details, thin lines, and point features. Convolution with a Laplacian kernel emphasizes high-frequency components, and sharpening is achieved by combining this response with the original image as:

$$\text{Sharpened Image} = \text{Original Image} - \alpha \cdot \text{Laplacian}(\text{Image})$$

or

$$I_{\text{sharp}} = I - \alpha \nabla^2 I$$

where α controls the sharpening strength and the following kernels are used to get the value of $\nabla^2 I$.

4-connected Laplacian:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

8-connected Laplacian:

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

12.2 Unsharp Masking

Unsharp masking is a classical image sharpening technique that enhances fine details by amplifying high-frequency components extracted from a blurred version of the image. The following steps need to be followed for the algorithm.

1. Generate a blurred version of the original image, typically using Gaussian smoothing.
2. Compute the detail mask by subtracting the blurred image from the original image.
3. Add a scaled version of this mask to the original image to obtain the sharpened result, where the scaling factor controls sharpening strength.

13 Key observations

- Gaussian blurring acts as a low-pass filter, suppressing high-frequency components associated with noise, fine details, and sharp edges. In the magnitude spectrum, this results in attenuation of high-frequency content while preserving low-frequency components.
- Image sharpening behaves as a high-pass filter, enhancing intensity variations and amplifying edges and textures. This is reflected in the magnitude spectrum as increased emphasis on high-frequency components.
- Edge detection strongly emphasizes high-frequency information corresponding to abrupt intensity transitions, producing a magnitude spectrum dominated by high-frequency responses.

14 Image Gradients and Corners

14.1 Theory

To understand local image structure, we analyze how pixel intensities change in a small neighborhood around a pixel. If we shift a small patch in different directions:

- In a **flat region**, the patch looks almost the same in all directions.

- Along an **edge**, the patch changes significantly only in the direction perpendicular to the edge.
- At a **corner**, the patch changes in all directions.

Hence, corners are highly informative features because they represent strong intensity variation in multiple directions.

14.2 Mathematical Formulation

Image gradients are used to quantify intensity changes:

$$I_x = \frac{\partial I}{\partial x}, \quad I_y = \frac{\partial I}{\partial y}$$

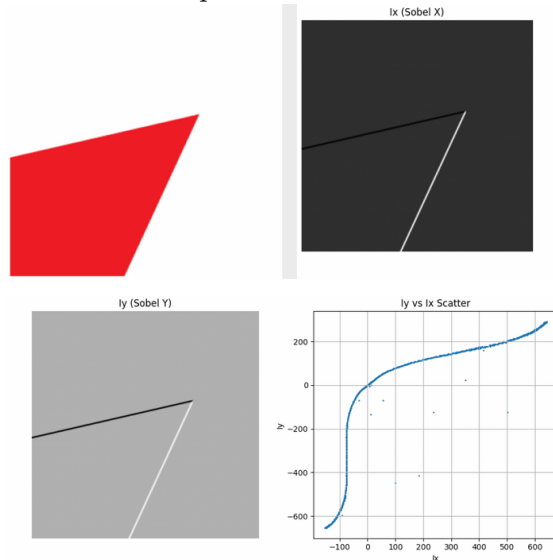
The gradients are commonly computed using Sobel operators. The distribution of (I_x, I_y) values helps distinguish between flat regions, edges, and corners.

14.3 Visual Results

The following images show the variation of

I_x and I_y

occurs at the points around the corners



15 Color Spaces

15.1 RGB Histogram

In the RGB color space, an image is represented using three channels: Red, Green, and Blue. A separate histogram for each channel shows the intensity distribution.

- High Red values usually indicate a warm image.
- High Blue values indicate a cooler tone.
- Unequal peaks suggest color imbalance.

15.2 HSV Color Space

HSV (Hue, Saturation, Value) is a perceptually intuitive color space.

- **Hue:** Represents the pure color and is measured as an angle.
- **Saturation:** Indicates color vividness.
- **Value:** Represents brightness.

HSV separates color information from illumination, making it useful for color-based segmentation tasks.

15.3 HSL Color Space

HSL (Hue, Saturation, Lightness) differs from HSV mainly in the definition of brightness. Lightness takes the midpoint between black and white, making saturation strongest at $L = 0.5$.

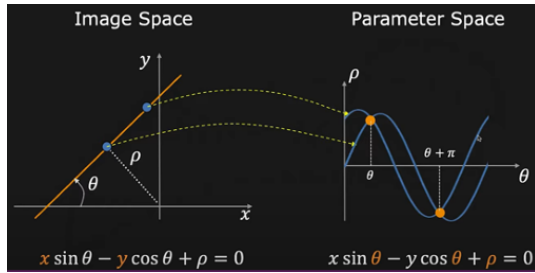
16 Hough Transform

16.1 Line Detection

Traditional edge detectors like Sobel or Canny focus only on local gradient information. In contrast, the Hough Transform incorporates prior knowledge about object

shape. A line in image space is represented as:

$$y = mx + c$$

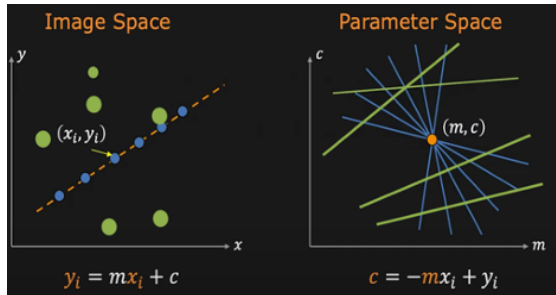


In the Hough Transform, a point in image space maps to a line in parameter space (m, c) . Multiple collinear points intersect at a common point in parameter space, indicating the presence of a line.

16.2 Alternative Parameterization

To avoid issues with infinite slopes, the normal form is used:

$$x \cos \theta + y \sin \theta = \rho$$



Here, (ρ, θ) form a finite and stable parameter space.

16.3 Algorithm (Line Hough Transform)

- Initialize an accumulator array in parameter space.
- For each edge pixel, vote for all possible (ρ, θ) .
- Find peaks in the accumulator.

- Map these peaks back to lines in image space.

17 Circle and Ellipse Detection

17.1 Circle Detection

For a circle with center (a, b) and radius r :

$$(x - a)^2 + (y - b)^2 = r^2$$

If r is known, the accumulator is two-dimensional. If r is unknown, the parameter space becomes three-dimensional, increasing computational complexity. Gradient information helps reduce this cost.

17.2 Ellipse Detection

Ellipse detection further increases complexity as five parameters are involved: center coordinates, major axis, minor axis, and orientation. This makes ellipse detection computationally expensive and sensitive to noise.



18 Brightness, Contrast, and Saturation

18.1 Brightness

Brightness refers to the overall luminance of an image. Increasing brightness is equiv-

alent to adding a constant β :

$$I_{new} = I_{old} + \beta$$

This shifts the histogram but does not improve contrast.

18.2 Contrast

Contrast enhances differences between light and dark regions:

$$I_{new} = \alpha I_{old}$$

$\alpha > 1$ increases contrast, while $\alpha < 1$ reduces it.

18.3 Saturation

Saturation controls color intensity. High saturation produces vivid colors, while low saturation results in grayscale-like images. Over-saturation may cause channel clipping.

19 Key Observations and Takeaways

- Corners are strong features because intensity varies in all directions.
- HSV and HSL color spaces are more aligned with human perception than RGB.
- Hough Transform is powerful for detecting parametric shapes like lines and circles.
- Increasing dimensionality of the parameter space increases memory and computation cost.
- Brightness shifts histograms, contrast stretches them, and saturation controls color vividness.

20 Feature Engineering and first group assignment

In the final session of the course, we learned how to combine all the computer vision concepts we studied into a single, functional pipeline. This lecture focused on using traditional computer vision techniques on a real dataset and turning visual information into numerical features for a learning model.

We used the Fruits-360 dataset, which has images of fruits taken under controlled conditions. The task was set up as a classification problem. The goal was to predict the fruit label using both raw images and well-designed features.

The main idea of this lecture was:

More meaningful inputs lead to better outputs.

Rather than just using raw pixel values, we extracted **color features**, **shape features**, and **texture features**. Each type of feature captures a different visual aspect of the fruit.

20.1 Color Features

We extracted color features in the HSV color space, which separates color identity from brightness and lighting effects. We used six linear color features, all derived from the HSV channels.

20.1.1 Mean Hue

The mean hue shows the dominant color of the fruit:

$$\mu_H = \frac{1}{N} \sum_{i=1}^N H_i$$

Intuition: Hue represents color identity as an angle on the color wheel. Fruits typically have characteristic hues:

- Banana → yellow-green
- Apple → red or green
- Orange → orange

So, mean hue helps us tell fruits apart based on their main color.

20.1.2 Standard Deviation of Hue

$$\sigma_H = \sqrt{\frac{1}{N} \sum_{i=1}^N (H_i - \mu_H)^2}$$

Intuition: This shows how consistent the color is across the surface of the fruit.

- Fresh, uniformly colored fruits have low σ_H
- Spotted or textured fruits have higher σ_H

20.1.3 Mean Saturation

$$\mu_S = \frac{1}{N} \sum_{i=1}^N S_i$$

Intuition: Saturation indicates color purity.

- Fresh fruits usually have high saturation
- Overripe or dull fruits typically have lower saturation

20.1.4 Standard Deviation of Saturation

$$\sigma_S = \sqrt{\frac{1}{N} \sum_{i=1}^N (S_i - \mu_S)^2}$$

Intuition: This captures variation in color purity.

- Smooth fruits → uniform saturation
- Rough or uneven surfaces → higher variation

20.1.5 Mean Value

$$\mu_V = \frac{1}{N} \sum_{i=1}^N V_i$$

Intuition: Value relates to brightness.

- Helps distinguish dark berries from bright fruits
- Important because the dataset lighting was controlled

20.1.6 Standard Deviation of Value

$$\sigma_V = \sqrt{\frac{1}{N} \sum_{i=1}^N (V_i - \mu_V)^2}$$

Intuition: This shows how shadows and highlights are spread out.

- Smooth spherical fruits show specular highlights
- Rough fruits show uneven lighting

20.2 Shape Features

Shape features were taken from the **largest contour** found in the edge-detected image. This ensured that the features described the fruit itself and not background noise.

20.2.1 Area Ratio

$$\text{Area Ratio} = \frac{A_{\text{contour}}}{A_{\text{image}}}$$

What it shows: Relative size of the fruit within the image frame.

Why normalized: All images are re-sized, making this ratio independent of scale.

Interpretation:

- Grapes → small area ratio
- Apples or melons → larger area ratio

20.2.2 Aspect Ratio

$$\text{Aspect Ratio} = \frac{w}{h}$$

where w is the width and h is the height of the bounding box.

What it shows: Elongation of the object.

Interpretation:

- Banana $\rightarrow$ high aspect ratio
- Apple $\rightarrow$ aspect ratio close to 1

This feature remains stable despite rotation and orientation changes.

moments describe mass distribution, image moments describe intensity distribution.

After centering around the centroid and normalizing for scale and rotation, we obtain seven Hu moments ϕ_1 to ϕ_7 .

20.2.3 Solidity

$$\text{Solidity} = \frac{A_{\text{contour}}}{A_{\text{convex hull}}}$$

What it shows: How compact or “filled” the object appears.

Interpretation:

- Smooth, round fruits $\rightarrow$ high solidity
- Indented or curved fruits $\rightarrow$ lower solidity

This feature does not get affected by small surface dents, making it very reliable.

20.3.1 Why Only the First Two Moments Were Used

We were specifically taught to use only the first two Hu moments:

- ϕ_1 : Overall shape spread
- ϕ_2 : Shape variance

Reason:

- Higher-order moments can be noisy
- Lower-order moments are more stable
- Compressing features is desirable

20.2.4 Circularity

$$\text{Circularity} = \frac{4\pi A}{P^2}$$

where A is area and P is perimeter.

What it shows: Overall roundness of the object.

Values:

- Circularity = 1 $\rightarrow$ perfect circle
- Circularity < 1 $\rightarrow$ elongated or irregular shape

This feature is especially useful for telling apples apart from bananas.

20.3.2 Interpretation of Hu Moments

ϕ_1 (**Overall Shape Spread**): Captures how spread out the shape is around its center.

- Compact shapes $\rightarrow$ lower spread
- Elongated shapes $\rightarrow$ higher spread

ϕ_2 (**Shape Variance**): Measures variation in shape distribution.

- Regular shapes $\rightarrow$ low variance
- Irregular shapes $\rightarrow$ higher variance

Using these two moments offers a compact yet effective representation of global shape.

20.3 Hu Moments

We were introduced to Hu Moments as global shape descriptors that stay consistent across:

- Translation
- Scale
- Rotation

The analogy explained in class was with **moment of inertia** in physics. Just as

20.4 Texture Features: Local Binary Patterns (LBP)

Texture was analyzed using Local Binary Patterns (LBP). The guiding question we used in class was:

“Is my neighbor brighter than me or not?”

For a center pixel g_c and its neighbors g_p , LBP is defined as:

$$\text{LBP} = \sum_{p=0}^{P-1} s(g_p - g_c) 2^p$$

where:

$$s(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

20.4.1 Intuition Behind LBP

LBP turns local texture patterns into binary codes.

- Flat regions create uniform codes
- Textured regions produce varied codes

20.4.2 Why LBP Is Useful

LBP has several important qualities:

- Strong against changes in lighting
- Sensitive to local texture
- Simple and quick to compute

Optionally, rotation-invariant LBP can be derived by minimizing the binary code.

20.4.3 LBP Histogram

Instead of using raw LBP values, we learned to create a histogram of LBP codes. This histogram captures the spread of texture patterns across the fruit surface.

20.5 Final Feature Pipeline Summary

In this session, the final input to the learning model included:

- Original image
- Edge-detected image (Canny)
- LBP image
- 6 color features
- 6 shape features

- 2 Hu moment features

This mix of raw visual data and carefully crafted features provided a rich representation of each fruit.

20.6 Key Takeaway from Session 5

The main lesson from this session was that traditional computer vision methods can create strong representations when combined thoughtfully, even before applying machine learning models.

20.7 Object-Oriented Programming (OOPS), Data Loader and first group assignment

During the course, we were taught the fundamentals of **Object-Oriented Programming (OOPS) in Python** and how these principles are essential for building scalable and modular deep learning systems. We learned that PyTorch heavily relies on object-oriented design to abstract complex functionality and promote code reusability.

We were taught core OOPS concepts such as classes, objects, inheritance, encapsulation, and method overriding, and we learned how these ideas translate directly into practical deep learning workflows. In particular, we learned that PyTorch's data handling pipeline is built around object-oriented abstractions in the form of the `Dataset` and `DataLoader` classes.

20.7.1 Dataset Class as an OOPS Abstraction

We were taught that the `Dataset` class represents a clear example of object-oriented abstraction. A `Dataset` encapsulates all logic related to how raw data is accessed, processed, and converted into a form suitable for a learning network.

By creating a custom Dataset class, we learned how to define:

- how data samples are indexed and retrieved,
- how individual inputs (such as images) are loaded from disk,
- how preprocessing and transformations are applied,
- how labels are associated with each input sample.

We learned that this is achieved by overriding key methods such as `__init__()`, `__len__()`, and `__getitem__()`, which is a direct application of OOPS concepts like method definition and encapsulation. Each Dataset object thus behaves as a well-defined interface between raw data and the learning algorithm.

Through this approach, we were taught that data preprocessing operations such as resizing, normalization, augmentation, and tensor conversion can be applied dynamically at runtime. This avoids redundant storage of modified data and ensures flexibility in experimentation.

Overall, we learned that the Dataset class defines **what data** is provided to the learning network and **how each individual sample** is prepared before training.

20.7.2 DataLoader Class and Controlled Data Flow

We were taught that while the Dataset class handles sample-level operations, the `DataLoader` class is responsible for managing how data is fed into the learning network during training and evaluation.

The `DataLoader` wraps around a Dataset object and applies higher-level data handling operations such as:

- grouping samples into mini-batches,
- shuffling data to improve generalization,

- loading data efficiently using parallel workers,
- delivering batches in a format compatible with the network input.

We learned that the `DataLoader` retrieves samples from the Dataset, applies batching logic, and outputs tensors with consistent shapes and dimensions. This ensures that the learning network receives input data in an organized and efficient manner, which is crucial for stable and effective training.

Importantly, we were taught that the `DataLoader` allows modifications to how data is supplied to the network without altering the Dataset itself. For example, changes in batch size, shuffling behavior, or data loading strategy can be made at the `DataLoader` level, demonstrating the benefit of separation of concerns enabled by OOPS.

20.7.3 Integration Through OOPS and Practical Application

We learned that the interaction between the Dataset and `DataLoader` classes demonstrates the power of object-oriented design. The Dataset focuses on data representation and preparation, while the `DataLoader` focuses on data delivery and iteration.

As part of the group assignment involving the **Fruits-360 dataset**, we were taught to implement a custom Dataset class using OOPS principles and then integrate it with a `DataLoader` to feed data into the learning network. This practical exercise helped us understand how object-oriented abstractions simplify complex data pipelines.

Overall, this learning experience reinforced how OOPS in Python enables clean, modular, and scalable deep learning work-

flows, and how PyTorch's Dataset and DataLoader classes form a crucial link between raw data and the learning network.

21 Unsupervised Learning: K-Means Clustering

21.1 Problem Formulation

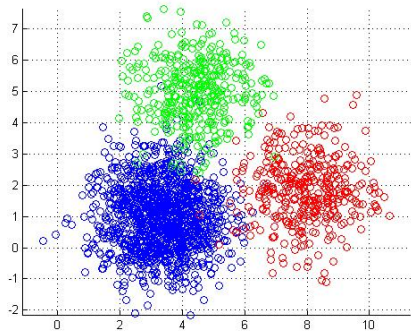
K-Means clustering is an unsupervised learning algorithm that aims to discover structure in unlabeled data by grouping samples into K clusters. The objective is to partition the dataset such that samples within the same cluster are more similar to each other than to those in other clusters.

21.2 Objective Function

The clustering objective is formulated as minimization of the within-cluster sum of squared distances:

$$\mathcal{L} = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

where μ_k denotes the centroid of the k -th cluster.



21.3 Algorithmic Procedure

The algorithm proceeds iteratively through alternating steps of cluster assignment

based on minimum Euclidean distance and centroid update via mean computation, until convergence.

21.4 Initialization Sensitivity and K-Means++

A major limitation of standard K-Means is its sensitivity to random centroid initialization, which can result in suboptimal convergence. K-Means++ addresses this by initializing centroids using a distance-based probability distribution, ensuring better separation among initial cluster centers.

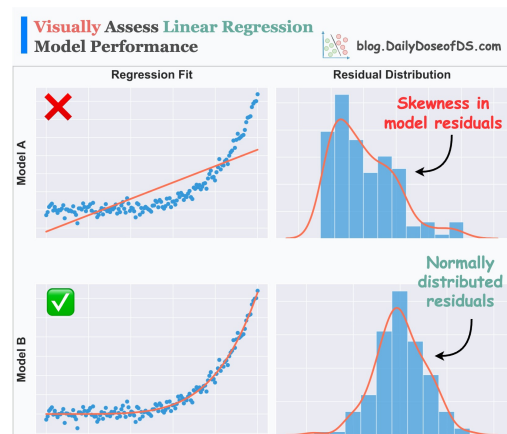
22 Supervised Learning: Linear Regression

22.1 Model Assumptions

Linear regression assumes a linear relationship between input features and the target variable, expressed as:

$$\hat{y} = \mathbf{w}^T \mathbf{x}$$

Additionally, it is assumed that the noise in observations follows a Gaussian distribution with zero mean and constant variance.



22.2 Loss Function and Maximum Likelihood Estimation

Under the Gaussian noise assumption, parameter estimation via Maximum Likelihood Estimation leads to minimization of the Mean Squared Error (MSE):

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T x_i)^2$$

22.3 Gradient-Based Optimization

The gradient of the MSE loss with respect to model parameters is given by:

$$\nabla \mathcal{L} = \frac{2}{N} X^T (X\mathbf{w} - \mathbf{y})$$

which enables optimization using gradient descent.

22.4 Overfitting and Polynomial Feature Expansion

Although linear in parameters, linear regression can model nonlinear functions through polynomial feature mappings. Excessive feature expansion, however, can lead to overfitting, where training error approaches zero while generalization deteriorates.

22.5 Regularization

To control model complexity, L2 regularization is introduced:

$$\mathcal{L} = \text{MSE} + \lambda \|\mathbf{w}\|^2$$

This penalizes large weights and improves generalization. Regularization is explained in more detail further under the section 24.4.3

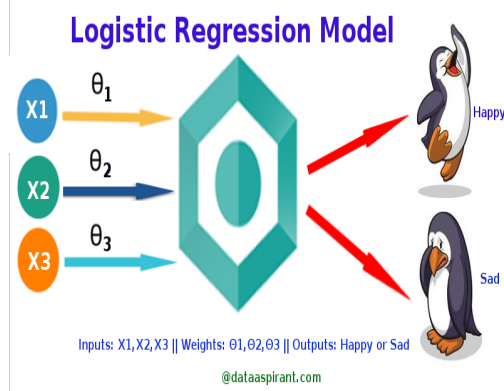
23 Supervised Learning: Logistic Regression

23.1 Classification Setting

Logistic regression is a statistical classification model used to estimate the probability of class membership rather than predicting continuous output values directly. In binary classification, it models the probability that an input belongs to a particular class by applying the logistic (sigmoid) function to a linear combination of input features, ensuring that the predicted probabilities lie in the range $[0,1]$.

Beyond binary classification, logistic regression can be extended to handle multi-class classification problems. In such cases, the model estimates the probabilities of an input belonging to each of multiple mutually exclusive classes. This is commonly achieved using the softmax function, which generalizes the sigmoid function to multiple classes and produces a valid probability distribution whose components sum to one. Alternatively, multiclass logistic regression can also be implemented using a one-vs-rest (OvR) strategy, where multiple binary logistic regression models are trained independently.

Overall, logistic regression is a probabilistic discriminative model that learns linear decision boundaries in the feature space while providing interpretable probability estimates. Its simplicity, efficiency, and strong theoretical foundation make it a widely used baseline method for both binary and multiclass classification tasks.



23.2 Sigmoid Activation

In logistic regression, the model first computes a linear predictor as a weighted sum of input features, represented by $\mathbf{w}^T x$. This linear combination can take any real value in $(-\infty, \infty)$. To interpret the output probabilistically, a nonlinear transformation is required.

The sigmoid activation function maps the linear predictor to a valid probability value in the range $[0, 1]$. This ensures that the model output can be interpreted as the conditional probability of the positive class given the input features:

$$P(y = 1|x) = \sigma(\mathbf{w}^T x), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

The sigmoid function is monotonic and differentiable, which allows efficient optimization using gradient-based methods. Its shape also introduces a smooth decision boundary, with values close to 0 or 1 indicating high confidence predictions, and values near 0.5 corresponding to uncertainty.

23.3 Likelihood and Loss Function

Logistic regression models the target variable as a Bernoulli random variable. Given a predicted probability $p = P(y = 1|x)$,

the likelihood of observing the true label $y \in \{0, 1\}$ is expressed using the Bernoulli distribution.

To estimate the model parameters, the likelihood is maximized over the training data. In practice, this is equivalent to minimizing the negative log-likelihood, which yields the binary cross-entropy loss function:

$$\mathcal{L} = -[y \log(p) + (1 - y) \log(1 - p)]$$

This loss function strongly penalizes confident but incorrect predictions, encouraging the model to assign high probability to the correct class. Its convexity with respect to the model parameters ensures the existence of a unique global optimum, making logistic regression stable and reliable during training.

24 MLP and PyTorch Library

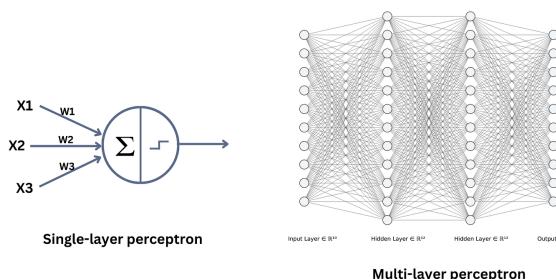
24.1 Definition and Dimensionality

At the core of modern deep learning is the **tensor**. A tensor is the main mathematical tool for representing data, model parameters, and intermediate calculations. You can think of a tensor as an extension of scalars, vectors, and matrices to any number of dimensions. This allows neural networks to manage structured, high-dimensional data easily.

From a mathematical point of view, tensors offer a common language for expressing linear algebra operations, multilinear mappings, and geometric changes. From a computational angle, tensors enable these operations to run efficiently on current hardware like GPUs.

In PyTorch, everything involved in training a neural network—such as inputs, weights, biases, activations, gradients, and

losses—is a tensor. This consistent representation allows automatic differentiation, hardware acceleration, and model composition to work smoothly.



0D Tensor (Scalar)

A single number.

Example: A loss value.

- **1D Tensor (Vector):** A vector is a one-dimensional tensor:

$$\mathbf{v} \in R^n$$

Vectors often represent feature vectors, bias terms, or flattened data samples. Geometrically, a vector indicates a point or direction in an n -dimensional space.

Formally, a tensor is part of a multidimensional real-valued space:

$$\mathcal{T} \in R^{d_1 \times d_2 \times \dots \times d_n}$$

Each dimension (or axis) represents a specific structural or semantic detail of the data. The number of dimensions in a tensor determines how data is organized and how mathematical operations are performed.

- **0D Tensor (Scalar):** A scalar is a tensor of rank zero:

$$\ell \in R$$

Scalars represent single numerical quantities like loss values, learning rates, or individual parameters. In optimization, the loss function is a scalar-valued function that guides learning through its gradients.

1D Tensor (Vector)

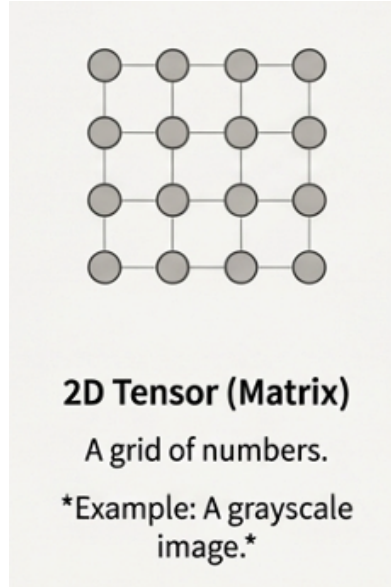
A sequence of numbers.

Example: A word embedding.

- **2D Tensor (Matrix):** A matrix is a two-dimensional tensor:

$$\mathbf{W} \in R^{m \times n}$$

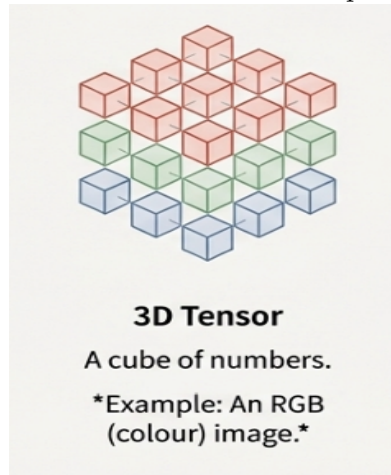
Matrices show linear transformations between vector spaces. In neural networks, weight matrices determine how input features combine to create activations.



- **3D Tensor:**

$$R^{H \times W \times C}$$

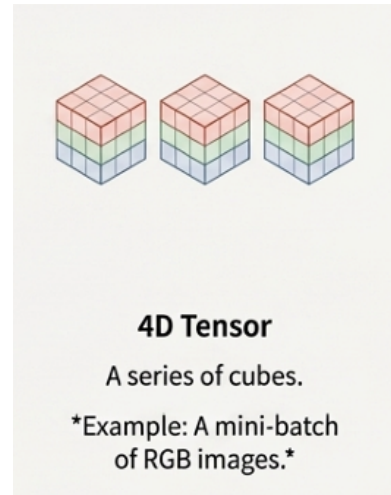
Three-dimensional tensors often represent structured data like color images, where height and width show spatial dimensions and channels represent different feature maps.



- **4D Tensor:**

$$R^{B \times C \times H \times W}$$

Four-dimensional tensors represent mini-batches of data, enabling multiple samples to be processed at once. This batching is crucial for effective gradient computation and stable optimization.



By organizing data and computations in this way, tensors help neural networks represent high-dimensional geometry, linear algebra, and nonlinear transformations consistently.

24.2 The Hardware Advantage: GPU Acceleration

The true power of tensors lies not just in their mathematical clarity but also in how well they work on modern computing hardware.

Traditional **CPU computation** is set up for sequential execution, processing one instruction at a time using complex control logic. Meanwhile, **Graphics Processing Units (GPUs)** are made for massive parallelism; they can perform thousands of simple arithmetic operations at the same

time.

Operations like matrix multiplication:

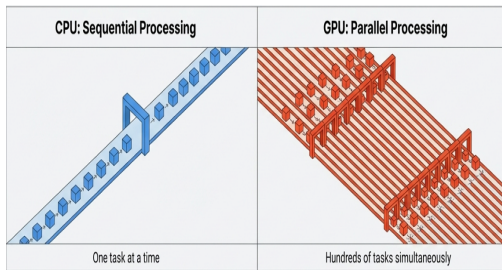
$$C_{ij} = \sum_k A_{ik} B_{kj}$$

are naturally parallel since each output can be calculated independently. This makes tensor operations a great match for GPU acceleration.

Deep learning models routinely do calculations such as:

$$\mathbf{Z} = \mathbf{W}\mathbf{X} + \mathbf{b}$$

Here, large matrices are multiplied across multiple layers and training steps. GPUs speed up these tasks by spreading computations across many processing cores, significantly cutting down training time.



Mathematical implication: The ability to train deep neural networks at scale comes from the connection between tensor algebra and parallel hardware.

24.2.1 The “Impossible Calculus” Problem

Training a neural network can be framed as a continuous optimization issue:

$$\min_{\theta} \mathcal{L}(\theta)$$

where θ represents all the trainable parameters, including weights and biases.

Calculating derivatives for simple func-

tions like

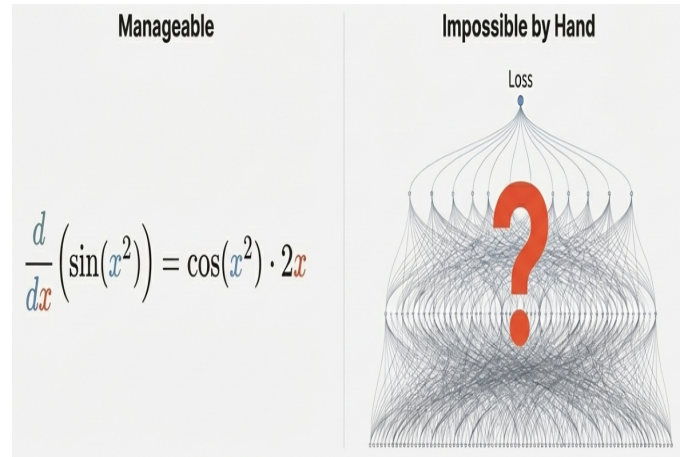
$$\frac{d}{dx} \sin(x^2) = 2x \cos(x^2)$$

is easy, but deep neural networks involve complex layers of linear transformations and nonlinear activation functions. As the depth and number of parameters increase, manually calculating gradients becomes difficult.

The key requirement for learning is to calculate:

$$\frac{\partial \mathcal{L}}{\partial w_i} \quad \forall w_i \in \theta$$

accurately and efficiently so that gradient-based optimization methods can update parameters.



24.2.2 PyTorch vs TensorFlow: Computational Graph Design

Deep learning frameworks vary mainly in how they create and run computational graphs.

TensorFlow usually uses a static graph method, where the whole computation graph is set up before running it. This allows for strong optimization for deployment but limits flexibility during testing.

PyTorch uses a dynamic graph method, building the computational graph as you

run Python code. This allows model structures to adjust on the fly and makes debugging easier.

- Graphs are formed during execution
- Native Python control flow is fully supported
- Errors can be checked immediately

Key takeaway: PyTorch focuses on flexibility and clarity, making it a good choice for research and learning.

24.2.3 Dynamic Computational Graphs in PyTorch

PyTorch calculates gradients using **dynamic computational graphs**, which are formally known as Directed Acyclic Graphs (DAGs).

- **Leaf nodes:** Trainable parameters like weights and biases
- **Intermediate nodes:** Results of operations like matrix multiplication, addition, and activation
- **Root node:** Final scalar loss value

During the forward pass, PyTorch tracks every tensor operation, creating a graph that captures the relationships between parameters and the loss.

24.2.4 Backward Pass and the Chain Rule

Once the loss is computed, calling `loss.backward()` starts the reverse-mode automatic differentiation.

Using the chain rule:

$$\frac{d\mathcal{L}}{dw} = \frac{d\mathcal{L}}{dZ} \cdot \frac{dZ}{dA} \cdot \frac{dA}{dw}$$

The autograd engine moves through the graph from the loss node back to the parameter nodes, gathering gradients along all valid paths.

Key insight: Backpropagation turns high-dimensional calculus into a graph traversal task.

24.2.5 Controlling Gradient Computation in PyTorch

PyTorch gives you clear control over gradient tracking:

- `requires_grad=True`: Turns on gradient tracking for parameters
- `torch.no_grad()`: Turns off gradient computation during inference
- Gradients build up by default and need to be reset manually

24.3 Multi-Layer Perceptrons (MLPs)

24.3.1 Limitations of Linear Models

A linear model is defined as:

$$y = \mathbf{w}^\top \mathbf{x} + b$$

This equation outlines a hyperplane:

$$\mathbf{w}^\top \mathbf{x} + b = 0$$

Linear models cannot capture non-linearly separable patterns like XOR or concentric rings.

Limitation: Linear decision boundaries lack curvature and flexibility.

24.3.2 Hidden Layers and Representation Learning

Hidden layers allow neural networks to learn complex representations.

Each hidden layer calculates:

$$\mathbf{h} = \phi(\mathbf{W}\mathbf{x} + \mathbf{b})$$

These transformations gradually reshape the input space, making it easier to find

complex decision boundaries.

24.3.3 Activation Functions

Activation functions add non-linearity.

- **Sigmoid:** Gives a probabilistic interpretation but suffers from vanishing gradients
- **Tanh:** Centered at zero but can saturate
- **ReLU:**

$$\phi(z) = \max(0, z)$$

ReLU networks can approximate complex functions using piecewise linear segments.

24.3.4 The Learning Loop in MLPs

Training involves an iterative optimization cycle that includes forward propagation, loss calculation, backward propagation, and parameter updates.

This process corresponds to moving down a high-dimensional loss surface.

24.4 Overfitting and Regularization in Neural Networks

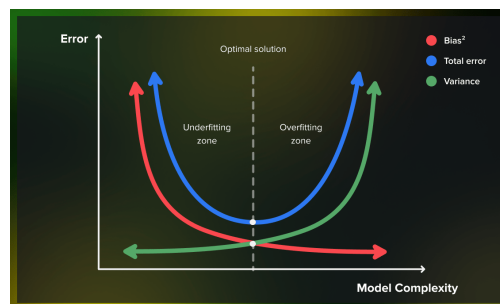
24.4.1 The Overfitting Problem

Overfitting happens when a model learns the training data by heart instead of grasping general patterns.

24.4.2 Bias–Variance Tradeoff

The bias–variance tradeoff describes the fundamental balance between a model’s ability to fit the training data and its ability to generalize to unseen data. The expected prediction error of a model can be decomposed as:

$$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

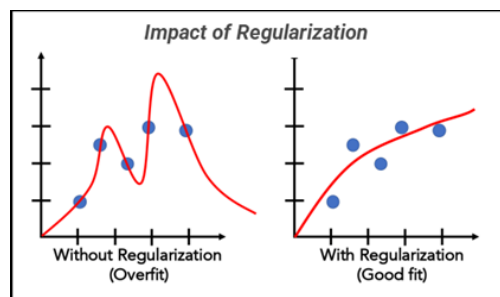


Bias measures the error introduced by approximating a complex real-world problem with a simplified model. High-bias models tend to underfit the data, failing to capture important patterns. Variance measures the sensitivity of the model to fluctuations in the training data; models with high variance often overfit, capturing noise rather than true structure. The noise term represents irreducible error inherent in the data due to measurement errors or stochasticity.

An effective learning algorithm seeks a balance between bias and variance to minimize the total prediction error, rather than optimizing either term in isolation.

24.4.3 Regularisation

Regularisation techniques are used to control model complexity and prevent overfitting by introducing a penalty on the model parameters. By discouraging excessively large weights, regularisation improves generalization performance on unseen data. Two commonly used regularisation methods are L1 and L2 regularisation.



L1 Regularisation (Lasso) L1 regularisation adds an absolute value penalty on the model weights to the loss function:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \sum_i |w_i|$$

This form of regularisation encourages sparsity in the model parameters by driving some weights exactly to zero. As a result, L1 regularisation performs implicit feature selection, making the model simpler and more interpretable. However, it may lead to less stable solutions when features are highly correlated.

L2 Regularisation (Weight Decay) L2 regularisation, also known as weight decay, penalizes the squared magnitude of the model parameters:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \sum_i w_i^2$$

The regularisation parameter λ controls the strength of the penalty. Larger values of λ enforce stronger shrinkage of the weights toward zero, discouraging overly complex models. By constraining the magnitude of the weights, L2 regularisation reduces model variance while slightly increasing bias, thereby improving generalization performance.

In gradient-based optimization, L2 regularisation results in a proportional decay of weights at each update step, which helps stabilize training and prevents extreme parameter values.

24.5 Threat Zone Finding Assignment

The Threat Zone Finding assignment applied theoretical concepts to a practical problem.

An MLP was built from scratch, offering insight into model structure and training dynamics.

PyTorch's automatic differentiation was used explicitly to track gradient flow.

Techniques for feature engineering were employed to enhance learning.

Overall, the assignment strengthened the link between theory, implementation, and performance.

25 Convolutional Neural Networks

25.1 Convolutional Layers

25.1.1 Concept and Motivation

A convolutional layer extracts local spatial patterns from structured data like images. Unlike fully connected layers that treat each input dimension separately, convolutional layers take advantage of the strong correlation between nearby pixels in an image.

The main idea behind convolution is to learn small filters that identify meaningful patterns such as edges, corners, textures, or shapes, regardless of where they appear in the image.

This is done through three key principles:

- Local connectivity
- Parameter sharing
- Translation equivariance

These principles enable convolutional layers to be both efficient and expressive.

25.1.2 Mathematical Formulation

Let the input image be represented as a tensor:

$$\mathbf{X} \in \mathbb{R}^{H \times W}$$

Let a convolutional kernel (filter) be:

$$\mathbf{K} \in \mathbb{R}^{k \times k}$$

The convolution operation produces a feature map:

$$\mathbf{Y} = \mathbf{X} * \mathbf{K}$$

Each element of the output feature map is computed as:

$$Y_{i,j} = \sum_{u=1}^k \sum_{v=1}^k X_{i+u,j+v} \cdot K_{u,v}$$

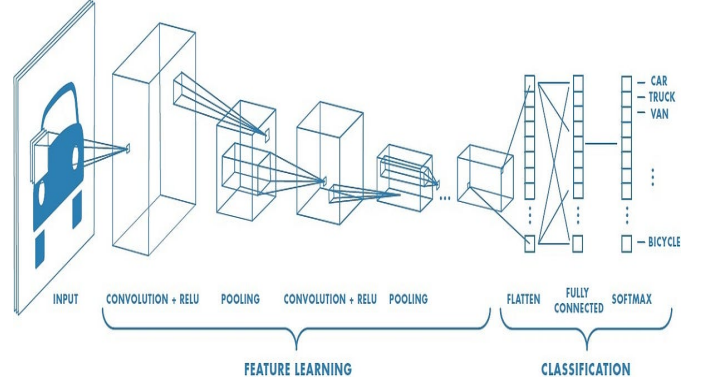
This operation is like calculating an inner product between the kernel and a local receptive field of the input.

25.1.3 Geometric and Computational Intuition

Each convolutional neuron responds well when the local input pattern matches the learned kernel. Since the same kernel is applied across the entire image, the layer learns to find the same feature at different locations.

From a linear algebra standpoint, convolution can be seen as multiplication by a structured, sparse matrix with shared parameters. However, this matrix is never built explicitly, making convolution computationally efficient.

Key insight: Convolutional layers are structured linear transformations followed by non-linear activation functions.



25.2 Pooling Operations

25.2.1 Purpose of Pooling

Pooling layers lower the spatial resolution of feature maps while keeping the most important information. They have three main purposes:

- Reduce computational complexity
- Increase the receptive field size
- Introduce robustness to small spatial shifts

Pooling layers do not have learnable parameters. Instead, they use a fixed aggregation function over local regions.

25.2.2 Mathematical Definition

Given an input feature map:

$$\mathbf{X} \in \mathbb{R}^{H \times W}$$

A pooling operation computes:

$$Y_{i,j} = \text{pool}(\{X_{u,v} \mid (u,v) \in \mathcal{R}_{i,j}\})$$

where $\mathcal{R}_{i,j}$ is a local pooling region.

For **max pooling**:

$$Y_{i,j} = \max_{(u,v) \in \mathcal{R}_{i,j}} X_{u,v}$$

For **average pooling**:

$$Y_{i,j} = \frac{1}{|\mathcal{R}_{i,j}|} \sum_{(u,v) \in \mathcal{R}_{i,j}} X_{u,v}$$

25.2.3 Intuition and Effect

Pooling summarizes local activations into a representative value. Max pooling highlights the strongest response, while average pooling captures the overall activity.

Important distinction:

- Convolution layers are translation equivariant
- Pooling layers provide partial translation invariance

Pooling can be seen as a form of non-linear downsampling that enhances generalization.

25.3 Fully Connected Layers

25.3.1 Role in Neural Networks

Fully connected (FC) layers are often used near the end of a neural network to perform global reasoning and decision-making. Unlike convolutional layers, FC layers connect every input unit to every output unit.

After convolution and pooling layers extract hierarchical features, fully connected layers combine these features to make final predictions.

25.3.2 Mathematical Formulation

Let the input vector be:

$$\mathbf{x} \in R^n$$

A fully connected layer computes:

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

where:

- $\mathbf{W} \in R^{m \times n}$ is the weight matrix
- $\mathbf{b} \in R^m$ is the bias vector

This operation represents a dense linear transformation followed by an optional non-linear activation.

25.3.3 Comparison with Convolutional Layers

- Fully connected layers use dense connections
- Convolutional layers use local connections and shared weights

Fully connected layers have many more parameters. This means a higher risk of overfitting, especially when applied directly to high-dimensional inputs like images.

Key insight: Convolutional layers extract structured features, while fully connected layers combine these features to create final predictions.

25.3.4 End-to-End Perspective

A typical CNN architecture follows this pattern:

- Convolutional layers: feature extraction
- Pooling layers: spatial abstraction
- Fully connected layers: classification or regression

Together, these layers form a pipeline that converts raw input data into high-level predictions.

References

- [1] Session Notes of lectures taken by mentors.
- [2] Fourier Series : (<https://www.youtube.com/watch?v=r6sGWTCMz2k>)
- [3] Fourier Transform : (<https://www.youtube.com/watch?v=spUNpyF58BY>)
- [4] DFT/FFT : (<https://www.youtube.com/watch?v=QmgJmh2I3Fw>)
- [5] Convolution : (<https://www.youtube.com/watch?v=KuXjwB4LzSA>)
- [6] Maths of Image Transform and DFT (<https://www.youtube.com/watch?v=2oEt5lbsyhM>)
- [7] More into FFT algorithm : (<https://www.youtube.com/watch?v=h7ap07q16V0>)
- [8] Intuition for Convolution : (<https://www.youtube.com/shorts/4xWpQe3G9qI>)
- [9] Binary Images : (<https://www.youtube.com/watch?v=C0xRYLrsscC>)
- [10] Geometric Properties : (<https://www.youtube.com/watch?v=ZPQiKXqHYrM>)
- [11] Segmenting Binary Images : (<https://www.youtube.com/watch?v=2ckNxewF5YU>)
- [12] Iterative Modification : (<https://www.youtube.com/watch?v=AYzWnCHnfSc>)
- [13] Edge Detection using Gradients : (<https://www.youtube.com/watch?v=10EBsQodtEQ>)
- [14] Corner and Edge Detection : (https://www.youtube.com/watch?v=Z_HwkG90Yvw)
- [15] OOPS in Python : (<https://youtube.com/playlist?list=PL-osiE80TeTsqhIu0QKhwlXsIBIdSeYtc&si=4SFIUbV2SgmjIEnx>)
- [16] Hough Transforms : (https://youtu.be/XRBc_xkZREg?si=5yP4tnP_eSF1A3Er)
- [17] Generalized Hough Transforms : (https://youtu.be/_mGxmZWs9Zw?si=paQoM363brizQ6By)
- [18] Active Contours : (<https://www.youtube.com/watch?v=FR0JUMk9P3Y>)
- [19] Linear Regression : (https://www.youtube.com/watch?v=4b4MUyve_U8&t=2s)
- [20] Logistic Regression : (<https://www.youtube.com/watch?v=het9HFqo1TQ&t=476s>)
- [21] K-Means Clustering : (<https://youtu.be/4b5d3muPQmA?si=mFGadTM4IYjj0dF2>)
- [22] K-Means Clustering : (<https://www.youtube.com/watch?v=yR7k19YBqiw&t=339s>)

- [23] MLP : (<https://youtu.be/aircAruvnKk>)
- [24] MLP : (https://d2l.ai/chapter_multilayer-perceptrons/index.html)
- [25] CNN : (<https://www.youtube.com/watch?v=py5by00HZM8>)
- [26] CNN : (https://d2l.ai/chapter_convolutional-neural-networks/index.html)
- [27] Ridge Regression : (<https://youtu.be/Q81RR3yKn30>)
- [28] Pytorch Tensors : (https://colab.research.google.com/drive/1fAANI5L6iHNRvjayxgwdXDu_aPkqxtu5?usp=sharing)
- [29] Autograd : (<https://colab.research.google.com/drive/1WJYgiriJgPzyMVyHJ8nPwA5fY-vikcXX#scrollTo=U8PrBEvQcLpF>)
- [30] Deep Neural Network Compression Research Paper : (<https://arxiv.org/pdf/1510.00149>)

THANK YOU